DAY 4 MORNING

JavaScript — Expanded Transcript-Based Study Notes

Detailed explanations • Sir’s code examples • Output reasoning • Interview definitions • Practice

How to use these notes

Follow each topic in this order: understand the idea, read the code, trace it line by line, predict the output, then practise explaining the interview definition aloud. The explanations expand the topics and examples in the existing Day 4 Morning notes. They stay focused on the same syllabus rather than adding unrelated topics.

Important: Code comments show expected output. Try predicting the output before reading the comment.

1. Nullish coalescing (??) and logical OR (||)

The lecture begins with a value that may be missing. Sometimes we want to use a default value such as `"hi"` when the original value is `null` or `undefined`. JavaScript provides the nullish coalescing operator `??` for this.

```
var obj = null;

var obj1 = obj === null || obj === undefined ? "hi" : obj;
console.log(obj1); // hi

// Shorter equivalent for this case:
var obj2 = obj ?? "hi";
console.log(obj2); // hi
```

Read the first expression from left to right: check whether `obj` is null OR undefined. If either condition is true, choose `"hi"`; otherwise keep the original value. The `??` operator expresses this default-value idea more directly.

What makes ?? different from ||?

```
console.log(null ?? "hi");       // "hi"
console.log(undefined ?? "hi");  // "hi"
console.log(0 ?? "hi");           // 0
console.log(false ?? "hi");       // false
console.log("" ?? "hi");          // ""

console.log(0 || "hi");            // "hi"
console.log(false || "hi");        // "hi"
console.log("" || "hi");           // "hi"
```

`??` checks only whether the left value is `null` or `undefined`. The `||` operator checks truthiness, so it also replaces values such as zero, false and an empty string.

Interview definition: The nullish coalescing operator (??) returns its right-hand operand when the left-hand operand is null or undefined; otherwise, it returns the left-hand operand.

Remember: If 0, false or an empty string is a valid value, ?? can preserve it while || will choose the fallback.

Quick output practice

Predict each result: `0 ?? 5`, `0 || 5`, `null ?? 5`, and `false ?? true`. Answers: 0, 5, 5, and false.

2. Arrays and array indexing

An array stores multiple values under one variable name. In JavaScript, values in an array do not all have to be of the same type. Arrays can also grow or shrink as elements are added or removed.

Interview definition: An array is an ordered collection of values accessed using zero-based numeric indexes.

```
var arr = [10, 20, 30, 40, "hello", true];

console.log(arr);        // prints the complete array
console.log(arr[0]);     // 10
console.log(arr[4]);     // "hello"
console.log(arr.length); // 6

for (var i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}
```

How to read the index

The first element is at index 0, not index 1. In this example, the indexes are 0, 1, 2, 3, 4 and 5. The array contains six elements, so `arr.length` is 6, but the last valid index is 5.

The loop begins with `i = 0`. Each round prints `arr[i]` and then increases `i`. The condition `i < arr.length` stops the loop when i reaches 6.

Remember: Do not use i <= arr.length for this loop. That condition tries to read arr[6], which is outside the array.

What is dynamic about JavaScript arrays?

An array is not fixed to a single declared length. Its length depends on the elements currently stored. JavaScript also allows mixed values, as shown by the numbers, string and boolean in the example.

3. Two-dimensional arrays (array of arrays)

A two-dimensional array contains inner arrays. It can be imagined as rows of values. In the lecture example, rows have different lengths, which is why the inner loop checks the length of the current row.

> **Interview definition:** A two-dimensional array is an array whose elements are themselves arrays, often used to represent rows and columns.

```
var arr = [
    [10, 20, 30],
    [40, 50, 60, 65],
    [70, 80]
];

console.log(arr.length); // 3 outer elements (rows)

for (var i = 0; i < arr.length; i++) {
    for (var j = 0; j < arr[i].length; j++) {
        console.log(arr[i][j]);
    }
}
```

The outer loop selects a row using `arr[i]`. The inner loop moves across the values in that row. `arr[i][j]` means: first select row i, then select element j from that row.

• `arr.length` counts the inner arrays, so here it is 3.

• `arr[i].length` counts the elements in the currently selected row.

• Because the rows contain 3, 4 and 2 values, respectively, the inner loop must use the current row's length.

Trace the loops

When i = 0, the inner loop prints 10, 20, 30. When i = 1, it prints 40, 50, 60, 65. When i = 2, it prints 70, 80. The inner loop finishes one row before the outer loop moves to the next row.

4. JavaScript objects and JSON

An object groups related information using property names and values. In an object literal, a colon separates each property name from its value, and commas separate properties.

> **Interview definition:** A JavaScript object is a collection of properties, where each property associates a key (property name) with a value.

```
var student = {
    id: 101,
    name: "Ramesh",
    marks: 75,
    phone: ""
};

console.log(student);
console.log(student.id);    // 101
console.log(student.name);  // Ramesh
console.log(student.marks); // 75
```

The expression `student.name` reads the value stored under the `name` property. The same dot notation is used for `id` and `marks`.

Objects can contain arrays and other objects

```
var student = {
    id: 101,
    name: "Rohan",
    marks: 73,
    phone: ["8876661", "65555"],
    address: {
        houseNo: "A-56",
        street: "JS Nagar"
    }
};

console.log(student.phone[0]);       // "8876661"
console.log(student.address.houseNo); // "A-56"
```

Here, `phone` is an array inside the student object. Therefore, access one phone number with `student.phone[index]`. The `address` property is another object, so its house number is accessed with `student.address.houseNo`.

Object vs JSON

> **Interview definition:** JSON (JavaScript Object Notation) is a lightweight text-based format used to represent and exchange structured data, commonly between a client and a server.

A JavaScript object is a value used by JavaScript code. JSON is a text format for representing data when it needs to be exchanged. They look similar, but they are not exactly the same.

• JavaScript object literals can use unquoted property names, such as `name: "Rohan"`.

• Strict JSON requires property names and string values to use double quotes.

• Strict JSON does not allow functions or the value undefined.

```
// JavaScript object literal
var person = { name: "Rohan", age: 25 };

// JSON text representation
{"name":"Rohan","age":25}
```

> **Remember:** The JSON example above is shown as text, not as a standalone JavaScript statement. In JavaScript code, the object literal and JSON text have different roles.

5. Optional chaining (?.)

Sometimes an object may not contain a property we expect. Reading a missing property itself usually gives undefined, but trying to read another property from that undefined value causes an error.

> **Interview definition:** Optional chaining (?.) safely accesses a property or method and stops the access chain if the value immediately before ?. is null or undefined.

```
var student = {
    id: 101,
    name: "Rohan"
};

console.log(student.address);          // undefined
// console.log(student.address.houseNo); // Error
console.log(student.address?.houseNo); // undefined
```

There is no `address` property in this object. Therefore `student.address` is undefined. The expression `student.address.houseNo` then attempts to read `houseNo` from undefined, which causes an error. With `?.`, JavaScript stops that part of the chain and returns undefined instead.

> **Remember:** If student itself might be null or undefined, use a protected chain such as student?.address?.houseNo. Protecting only address does not protect student.

6. Array of objects

An array of objects is useful when we have multiple records, such as a list of students. Each array element is one student object, and every object can have its own property values.

> **Interview definition:** An array of objects is an array in which each element is an object containing related properties.

```
var studentsList = [
    { id: 101, name: "Rohan", marks: 73,
      phone: ["8876661", "65555"] },
    { id: 102, name: "Manoj", marks: 83,
      phone: ["90000", "6995555", "865677"] },
    { id: 103, name: "Vikram", marks: 61,
      phone: ["8876661", "65555"] }
];

for (var i = 0; i < studentsList.length; i++) {
    console.log(studentsList[i].id,
                studentsList[i].name,
                studentsList[i].marks);

    for (var j = 0; j < studentsList[i].phone.length; j++) {
        console.log(studentsList[i].phone[j]);
    }
}
```

The outer loop visits each student. `studentsList[i]` means the current student object. Adding `.name`, `.marks` or another property accesses a field from that object.

The inner loop is needed because each student has an array of phone numbers. It prints all numbers for the current student before the outer loop moves on.

A common indexing pattern

```
studentsList[0]             // first student object
studentsList[0].name        // name of first student
studentsList[0].phone       // phone array of first student
studentsList[0].phone[1]    // second phone value
```

7. Object destructuring

Destructuring lets us extract values from an object into variables. It can make code shorter when several properties of the same object are needed.

> **Interview definition:** Object destructuring is syntax that extracts properties from an object into variables.

```
var product = {
    pid: 111,
    productname: "Keyboard",
    price: 7666,
    qty: 6,
    address: { houseNo: "B-45", state: "React Nagar" }
};

const { pid, productname, price, qty, address } = product;
const { houseNo, state } = address;

console.log(pid);          // 111
console.log(productname);  // Keyboard
console.log(price);        // 7666
console.log(qty);          // 6
console.log(houseNo, state);
```

The pattern on the left uses property names that match the object's property names. For example, `price` receives the value of `product.price`. The nested `address` object is extracted first, and then its `houseNo` and `state` properties are extracted.

Remember: In this example, variable names match property names. Renaming a property while destructuring uses extra syntax and is not required to understand this example.

Without destructuring vs with destructuring

```
// Repeated property access
console.log(product.pid);
console.log(product.price);

// Extract once, then use variables
const { pid, price } = product;
console.log(pid);
console.log(price);
```

8. Spread syntax (...)

Spread syntax uses three dots to expand the elements of an array or the properties of an object into another array or object. In the lecture examples, it is used to combine two objects and combine two arrays.

Interview definition: Spread syntax (...) expands an iterable's elements or an object's enumerable properties into a new array or object.

```
const o1 = { x: 34, y: 30 };
const o2 = { z: 50 };

const o3 = { ...o1, ...o2 };
console.log(o3); // { x: 34, y: 30, z: 50 }

const a1 = [10, 20, 30];
const a2 = [40, 50];

const a3 = [...a1, ...a2];
console.log(a3); // [10, 20, 30, 40, 50]
```

In `{ ...o1, ...o2 }`, the properties from both objects are placed into the new object. In `[...a1, ...a2]`, the elements from the two arrays are placed in order into a new array.

What if both objects have the same key?

```
const first = { colour: "blue", size: 10 };
const second = { colour: "red" };
const combined = { ...first, ...second };

console.log(combined); // { colour: "red", size: 10 }
```

When the same property appears more than once, the later value wins in this object literal. Here, the later spread supplies `colour: "red"`.

Remember: Spread creates a shallow copy. If a property contains a nested object or array, the nested value is not deeply cloned.

9. Classes, constructors, objects and this

A class describes the properties and methods that its objects can have. The `new` keyword creates an object from the class. The constructor runs during that creation and can initialize the new object.

Interview definition: A class defines the structure and behaviour of objects. A constructor is a special method called automatically when a new instance is created.

```
class Product {
    constructor(id, name) {
        this.id = id;
        this.name = name;
    }

    printDetails() {
        console.log(this.id, this.name);
    }
}
```

```
var obj1 = new Product(101, "Keyboard");
obj1.printDetails();

var obj2 = new Product(110, "Mouse");
obj2.printDetails();
```

When `new Product(101, "Keyboard")` runs, the values 101 and Keyboard are passed to the constructor parameters `id` and `name`. Then `this.id = id` stores the id on the new object, and `this.name = name` stores the name.

The two objects are separate. `obj1` has its own id and name, and `obj2` has its own id and name. Calling `obj1.printDetails()` runs the method for obj1.

What does this mean here?

Inside a normal class method, `this` refers to the object on which the method is called. So when `obj1.printDetails()` runs, `this.id` means `obj1.id`.

> **Remember:** Objects are not restricted to properties declared in the constructor; JavaScript can allow properties to be added later. The constructor is a clear place to initialize the properties an object is expected to have.

10. Inheritance, extends and super()

Inheritance allows a child class to use properties and methods from a parent class. JavaScript uses `extends` to establish the relationship between classes. In a child class constructor, `super()` calls the parent constructor.

> **Interview definition:** Inheritance allows a class to derive properties and methods from another class. In JavaScript, extends establishes class inheritance, and super() calls the parent class constructor.

```
class A {
    constructor() {
        this.x = 10;
    }

    show() {
        console.log("inside show x=", this.x);
    }
}

class B extends A {
    constructor() {
        super();
        this.y = 20;
    }
}

var bObj = new B();
console.log(bObj.y); // 20
console.log(bObj.x); // 10
bObj.show();         // inside show x= 10
```

`B extends A` means B inherits from A. When `new B()` is created, the constructor in B runs. `super()` invokes A's constructor, which initializes `x`. Then B initializes `y`.

Because B inherits the `show()` method, `bObj.show()` can call it even though the method is written in class A.

Common inheritance types

• **Single:** one child class inherits from one parent class.

• **Multilevel:** inheritance forms a chain, for example A → B → C.

• **Hierarchical:** multiple child classes inherit from the same parent.

• **Multiple:** one class inherits from multiple parent classes. JavaScript classes do not directly support extending multiple classes at once.

> **Remember:** In a derived class constructor, call super() before using this.

11. Callback functions

A callback is a function passed as an argument to another function. The receiving function can then call it. This can be used to control what happens inside a general function.

> **Interview definition:** A callback function is a function passed as an argument to another function and invoked by that function or by a later operation.

```
function show(arg) {
    console.log(arg);
    arg();
}

function demo() {
    console.log("inside demo");
}

show(demo);
```

In `show(demo)`, the function itself is passed to show. Inside show, `console.log(arg)` prints the function value, and `arg()` calls that function.

Passing a named function vs calling it

```
show(demo);   // passes the function to show
show(demo()); // calls demo immediately, then passes its return value
```

This distinction is important. A function name without parentheses refers to the function. Parentheses call the function. If the function does not return a value, calling it typically produces undefined, so `show(demo())` would not pass the demo function itself.

Anonymous callback

```
show(function() {
    console.log("inside demo");
});
```

This passes a function expression directly without giving it a separate name. The function can still be called inside show through `arg()`.

Callback receiving a value

```
function process(arg) {
    console.log("processing....", arg);
}

function start(arg) {
    console.log("start called");
    arg(100);
}

start(process); // processing.... 100
```

The function `process` is passed into `start`. Inside start, the parameter arg refers to process. Therefore `arg(100)` calls process with 100.

12. Functions as values and arrow functions

JavaScript functions can be stored in variables and passed to other functions. This is why they are often called first-class values in JavaScript.

> **Interview definition:** A first-class function is a function that can be assigned to a variable, passed as an argument, or returned from another function.

```
let x = function() {
    console.log("anonymous function called");
};

console.log(typeof x); // "function"
```

```
x();                    // anonymous function called
```

The function expression is assigned to x, so x stores a function. Writing `x()` invokes it. The result of `typeof x` is the string `"function"`.

Arrow function syntax

Interview definition: An arrow function is a concise function-expression syntax written using the => symbol.

```
let sum = (x, y) => {
    let z = x + y;
    return z;
};

console.log(sum(2, 7)); // 9

let sum2 = (x, y) => x + y;
console.log(sum2(2, 7)); // 9

let increment = n => n + 5;
console.log(increment(2)); // 7
```

The first arrow function uses braces and an explicit return because it has multiple statements. The second has one expression and no braces, so its result is returned implicitly. With one parameter, the parentheses around n may be omitted.

Remember: If an arrow function body uses braces and you want it to return a value, write return explicitly.

13. Array callback methods: forEach, filter, map and reduce

These methods process array elements by calling a function for each element. The important thing is to understand the different purpose and return value of each method.

forEach() — do something for every element

Interview definition: forEach() calls a callback once for each array element and is mainly used to perform an action for every element.

```
var arr = [10, 45, 32, 76, 45];

arr.forEach((value, index) => {
    console.log(value, index);
});
```

The first callback parameter, value, receives the current element. The second, index, receives its index. In each iteration, one element and its index are printed. forEach is commonly used for actions such as printing; it does not create a transformed array.

filter() — keep values that pass a test

Interview definition: filter() creates a new array containing the original elements for which the callback returns true.

```
var arr = [10, 45, 32, 76, 45, 61, 78];
var resultArray = arr.filter(value => value % 2 === 0);
console.log(resultArray); // [10, 32, 76, 78]
```

The expression `value % 2 === 0` checks whether a number is even. filter checks each element. If the expression is true, that original value is kept in the result array; if false, it is left out.

map() — transform each value

Interview definition: map() creates a new array by applying a callback to each element of the original array.

```
var arr = [10, 45, 32, 76, 45, 61, 78];
var rs = arr.map(value => value * 2);
console.log(rs); // [20, 90, 64, 152, 90, 122, 156]
```

map does not ask whether to keep or reject an element. It computes a new value for each element. Here, each number is multiplied by 2, so the result array has the same number of elements as the original.

reduce() — combine values into one result

> **Interview definition:** reduce() processes array elements with a callback to combine them into one accumulated result, such as a sum.

```
var arr = [10, 20, 30];
var total = arr.reduce((sum, value) => sum + value, 0);
console.log(total); // 60
```

The second argument, 0, is the initial accumulated value. The callback adds the current value to sum. The accumulation proceeds as 0 + 10 = 10, then 10 + 20 = 30, then 30 + 30 = 60.

Compare the four methods

• **forEach:** perform an action for each item; usually returns undefined.

• **filter:** keep items that pass a condition; returns a new array that may be shorter.

• **map:** transform every item; returns a new array with one result per original item.

• **reduce:** combine items into one accumulated result, such as a number or object.

> **Remember:** A useful memory aid: forEach = action, filter = select, map = transform, reduce = combine.

14. Synchronous and asynchronous processing

Synchronous code runs in sequence: the next statement waits for the current statement to finish. Asynchronous operations can schedule work to happen later, allowing the remaining synchronous code to continue while waiting.

> **Interview definition:** Synchronous execution runs statements in sequence. Asynchronous programming allows an operation to be scheduled so other work can continue while it is pending.

```
console.log("logic started");

setTimeout(function() {
    console.log("show called");
}, 2000);

console.log("logic end");
```

Expected order: first `logic started`, then `logic end`, and later `show called`. setTimeout schedules the callback to run after approximately 2000 milliseconds (2 seconds); it does not pause the whole program for two seconds.

Why does show called appear last?

• JavaScript runs the first console.log immediately.

• setTimeout schedules its callback for a later time.

• The next console.log runs without waiting for the timer.

• After the delay has elapsed and the callback can run, show called is printed.

> **Remember:** The delay passed to setTimeout is a minimum scheduling delay, not a guarantee that the callback executes at exactly that millisecond. A busy main thread can delay it further.

JavaScript is commonly described as single-threaded at the execution level, while asynchronous APIs and the event loop coordinate callbacks. A long-running synchronous task can still block other work on the main thread.

15. Interview definitions — quick revision

JavaScript: A high-level programming language commonly used to add behaviour and interactivity to web pages.

Dynamic typing: A language behaviour in which a variable can hold values of different types at different times without a fixed declared type.

Array: An ordered collection of values accessed using zero-based indexes.

Two-dimensional array: An array containing inner arrays, often used to model rows and columns.

Object: A collection of properties represented as key–value associations.

JSON: A text-based data interchange format used to exchange structured data.

Optional chaining: The ?. operator, which safely stops property access when the preceding value is null or undefined.

Nullish coalescing: The ?? operator, which chooses a fallback only for null or undefined.

Destructuring: Syntax for extracting values from arrays or properties from objects into variables.

Spread syntax: The ... syntax used to expand array elements or object properties into another array or object.

Class: A template defining the structure and behaviour of objects.

Constructor: A special class method that runs when a new object instance is created.

Inheritance: A mechanism by which a class derives properties and methods from another class.

Callback: A function passed to another function to be called by it or by a later operation.

First-class function: A function that can be assigned to a variable, passed as an argument, or returned.

Arrow function: A concise function-expression syntax that uses =>.

forEach: An array method that invokes a callback for each element.

filter: An array method that returns a new array of elements satisfying a test.

map: An array method that returns a new array containing callback results for each element.

reduce: An array method that combines elements into one accumulated result.

Synchronous execution: Execution in sequence where the next statement waits for the current operation.

Asynchronous programming: A programming approach that lets other work continue while an operation is pending.

16. Common mistakes and accuracy checks

• **Array boundary:** use i < arr.length. i <= arr.length attempts one index past the end.

• **typeof null:** JavaScript returns "object" because of a historical quirk. null is not an ordinary object.

• **JSON vs object literal:** JSON is text and has stricter syntax than a JavaScript object literal.

• **Optional chaining:** student.address?.houseNo protects against address being nullish; student?.address?.houseNo also protects against student being nullish.

• **Inheritance:** call super() before using this in a derived class constructor.

• **Array methods:** filter and map return new arrays; forEach is for actions; reduce combines values.

• **setTimeout:** schedules a callback; it does not pause the whole program.

• **Callbacks:** show(demo) passes a function; show(demo()) calls demo immediately.

• **Spread:** if the same object key appears in multiple spreads, the later value wins.

17. Practice questions

1. What does 0 ?? 5 return?

Answer: 0, because zero is not null or undefined.

2. What does 0 || 5 return?

Answer: 5, because zero is falsy.

3. What is the last valid index of an array with 6 elements?

Answer: 5.

4. What does student.phone[1] mean?

Answer: The second element of the phone array inside student.

5. What does student.address?.houseNo protect against?

Answer: It protects against address being null or undefined.

6. What does const { price } = product do?

Answer: It extracts product.price into a variable named price.

7. What is the difference between { ...o1, ...o2 } and [...a1, ...a2]?

Answer: The first combines object properties; the second combines array elements.

8. Why call super() in B's constructor?

Answer: To run the parent constructor before using this in the child constructor.

9. What is a callback?

Answer: A function passed as an argument to another function.

10. What does filter return?

Answer: A new array containing elements that pass the test.

11. What does map return?

Answer: A new array containing the callback result for each original element.

12. Which array method can calculate a sum?

Answer: reduce.

13. What is the output order for the simple setTimeout example?

Answer: logic started, logic end, show called.

14. Can a JavaScript class directly extend two classes at once?

Answer: No; JavaScript supports single class inheritance directly.

15. What is JSON commonly used for?

Answer: Exchanging structured data between systems, such as a client and server.

18. Coding practice

• Create an array containing numbers, a string and a boolean. Print every element and the array length.

• Create a two-dimensional array whose rows have different lengths. Print every value using nested loops.

• Create a student object containing a phone array and a nested address object. Print selected values.

• Use ?? and || with 0, false, null, undefined and an empty string. Write down the outputs.

• Use destructuring to extract product properties and nested address properties.

• Merge two objects and two arrays using spread syntax.

• Create a Product class with a constructor and a printDetails() method.

• Create class A and class B extends A. Use super() and test inherited properties and methods.

• Pass a named function and an anonymous function as callbacks.

• Use arrow functions with forEach, filter, map and reduce.

• Run setTimeout callbacks with different delays and predict the output order before executing.

Suggested study method

1. Read the explanation once. 2. Explain each line of code aloud. 3. Predict the output before running the program. 4. Run it in the browser console. 5. Answer the interview definition in two or three sentences. 6. Redo the practice questions without looking at the answers.